

Sharks SPA Homework

Load the Crew from an API

Goal

Your task is to connect the **About page** in the Sharks SPA to the local API.

Currently, the instructor cards are written manually inside `AboutPage.js`.

You must change the page so that:

1. Instructor information comes from the API.
2. JavaScript creates the instructor cards.
3. The router waits for the API response.
4. A loading message appears while waiting.
5. An error message appears if the API is unavailable.

بطريقة منظمة، بدل ما نكتب البيانات يدويًا داخل الصفحة SPA داخل API الهدف: نتعلم إزاي نستخدم

Required final result

When you open:

`http://localhost:8000/#/about`

The About page should:

- Load the crew from the API.
- Display all crew cards.
- Keep the same design.
- Show a loading message briefly.
- Show an error message if the API server is stopped.
- Work without reloading the whole website.

Important rules

- Work only on the About page and router.
- Do not modify Courses, Pricing, Contact, or Home.
- Do not add POST, PATCH, or DELETE requests.

- Do not install a framework.
 - Do not use React.
 - Do not copy the crew cards manually.
 - Do not remove the SPA router.
 - Add simple Arabic comments to explain your JavaScript.
-

Part 1 — Start the project

Open two terminals.

Terminal 1: Start the API

```
npm install  
npm run api
```

The API should run on:

```
http://localhost:3000
```

Test this URL in the browser:

```
http://localhost:3000/crew?_sort=order
```

You should see an array containing the crew members.

Terminal 2: Start the website

```
python -m http.server 8000
```

Open:

```
http://localhost:8000/#/about
```

Part 2 — Create the API file

Create this file:

```
js/api.js
```

Inside it, create:

```
const API_URL = "http://localhost:3000";
```

Then create and export an asynchronous function called:

```
getJSON
```

The function must:

1. Receive an `endpoint`.
2. Use `fetch()`.
3. Wait for the response.
4. Check `response.ok`.
5. Throw an error if the response is unsuccessful.
6. Convert the response to JSON.
7. Return the final data.

The function should be used like this:

```
const crew = await getJson("/crew?_sort=order");
```

Required Arabic comments

Add comments that explain:

- What `API_URL` means.
- What `fetch()` does.
- Why we check `response.ok`.
- Why we use `response.json()`.
- Why we return the data.

Do not copy API-request code into `AboutPage.js`. The request logic must stay inside `api.js`.

Part 3 — Update the router

Open:

```
js/router.js
```

The router currently renders pages immediately.

Change the route-rendering function so it can wait for API pages.

Requirements

1. Make the function asynchronous

Change the route-rendering function to use:

```
async
```

2. Show a loading state

Before rendering the page, put a simple loading message inside `#app`.

Example result:

```
Loading page...
```

You may create better HTML and CSS for it, but keep it simple.

3. Wait for the page

Instead of rendering the page immediately, use:

```
await route.render()
```

4. Add error handling

Use:

```
try {  
  // Render the page  
} catch (error) {  
  // Display an error  
}
```

If an error happens, the page should display:

```
Something went wrong.  
We could not load this page. Please try again.
```

Also print the real error in the console using:

```
console.error(error);
```

Required Arabic comments

Explain:

- Why the router needs `async`.
- Why it uses `await`.
- What `try` does.
- What `catch` does.
- Why the user sees a simple message while the developer sees the full console error.

Part 4 — Connect the About page

Open:

```
js/pages/AboutPage.js
```

Step 1: Import the API function

Import `getJSON` from:

```
../api.js
```

Step 2: Make AboutPage asynchronous

The page function must use:

```
async
```

Step 3: Request the crew

Inside `AboutPage`, request:

```
/crew?_sort=order
```

Save the returned array in a variable called:

```
crew
```

Step 4: Create one crew-card function

Create a separate function called:

```
createCrewCard
```

It should:

- Receive one crew member object.
- Return the HTML for one instructor card.
- Use the member's:
 - `name`
 - `role`
 - `bio`
 - `image`

Example function call:

```
createCrewCard(member);
```

Do not put all crew cards manually inside the page.

Step 5: Convert the array into cards

Use:

```
map()
```

to convert each crew member into a card.

Then use:

```
join("")
```

to combine all the cards into one HTML string.

Save the result in:

```
crewCards
```

Step 6: Render the cards

Inside the existing crew grid, remove the manually written instructor cards.

Render:

```
${crewCards}
```

The rest of the About page must remain unchanged.

Required Arabic comments

Explain:

- What the `crew` array contains.
- What one `member` object represents.
- What `map()` does.
- Why `join("")` is required.
- What `${member.name}` means.

Part 5 — Check the image paths

The API returns image paths.

Use the value from:

```
member.image
```

Do not write a different image path manually for every instructor.

Check that all instructor images load correctly.

Do not move the asset files during this task.

Part 6 — Test your work

You must complete all the following tests.

Test 1: Successful API request

1. Start the API.
2. Open `#/about`.
3. Confirm all crew cards appear.
4. Confirm the design still looks correct.
5. Confirm there are no console errors.

Test 2: Loading state

1. Open the Network tab.
2. Refresh the About route.
3. Confirm that a loading message appears before the page content.

It may appear very quickly on a local server.

Test 3: Error state

1. Stop the API server.
2. Refresh `#/about`.
3. Confirm the error message appears.
4. Confirm the whole SPA does not crash.
5. Start the API again.

Test 4: SPA navigation

Navigate between:

```
#/about  
#/courses  
#/  

```

Confirm:

- The browser does not perform a full page reload.

- The header and footer remain working.
- The mobile menu remains working.
- Back and Forward buttons remain working.

Test 5: API data change

Temporarily change one crew member's name inside `db.json`.

Refresh `#/about`.

Confirm the new name appears without changing `AboutPage.js`.

Return the original name after the test.

Bonus challenge

Add one new crew member to the `crew` array inside `db.json`.

The new object must contain:

```
{
  "id": "5",
  "order": 5,
  "name": "Coach Marina",
  "role": "Communication Skills",
  "bio": "Marina helps students speak with confidence and work together.",
  "image": "/assets/Navigator Julia.png"
}
```

After saving:

1. Refresh the About page.
 2. Confirm the fifth card appears automatically.
 3. Do not add new HTML manually.
-

Files you are allowed to modify

```
js/api.js
js/router.js
```

```
js/pages/AboutPage.js
db.json
```

You may also modify the About-page CSS only when a small adjustment is required for the cards.

Do not modify unrelated files.

Submission requirements

Before the next session, send:

1. The updated project or GitHub commit.
2. A screenshot of the About page with the API running.
3. A screenshot of the error state with the API stopped.
4. A screenshot of the crew response in the browser or Network tab.
5. Your answers to these questions:

Questions

1. What is the responsibility of `api.js`?
 2. Why did `AboutPage` become an `async` function?
 3. Why does the router use `await route.render()`?
 4. What does `map()` return?
 5. Why do we use `join("")`?
 6. What is the difference between the loading state and error state?
 7. Why is this structure better than writing `fetch()` in many files?
-

Completion checklist

- ☐ `js/api.js` exists.
- ☐ `getJson()` is exported.
- ☐ `fetch()` is not repeated inside `AboutPage.js`.
- ☐ The router is asynchronous.
- ☐ The router displays a loading state.
- ☐ The router catches API errors.
- ☐ `AboutPage` requests `/crew?_sort=order`.
- ☐ Crew cards are created with a function.
- ☐ Crew cards use `map()` and `join("")`.
- ☐ Hard-coded crew cards were removed.
- ☐ API images display correctly.
- ☐ Other SPA routes still work.
- ☐ Arabic comments were added.

- [] There are no console errors while the API is running.
 - [] The error state works while the API is stopped.
-

The main idea

In the old Client project, you learned how to call an API.

In this task, you are learning how to **organize API calls correctly inside an SPA**.